AWS Foundations

Hands-On Practical Guide

Mission 02: GO CLOUD

Document Information

Modules covered: 2 (IAM) | 3 (VPC & Networking) | 4 (CloudFormation & CI/CD)

Module 1 theory: Covered in Day 2 slides — no CLI hands-on possible

Audience: Sigma DataTech bootcamp participants

Lab duration: Approx. 3 – 4 hours (labs only)

Tools required: AWS account with AdministratorAccess | AWS CLI v2 | VS Code

Region: Use ap-south-1 (Mumbai) throughout — nearest to training location

Before Every Lab Session

1. Log in to the AWS console with your assigned IAM user — NOT the root account.
2. Confirm your region is set to ap-south-1 (Mumbai) in the top-right corner.
3. Open AWS CloudShell (icon in the console header) or your local terminal with AWS CLI.
4. Verify CLI access: aws sts get-caller-identity

   Expected output shows your AccountId, UserId, and Arn.
5. Tag EVERY resource you create with: Project=sigma Owner=<your-name>

   This is how we track and clean up resources after the lab.

Module 2 — IAM & Access Control

IAM is the foundation of everything in AWS. Every API call, every service interaction, every S3 read and EC2 launch goes through IAM. Get this right and everything else is easier.

Estimated lab time: 45 minutes across 4 labs.

2.1 Lab — Users, Groups, and Password Policy

> **LAB 2.1 — Create IAM Users and Assign to Groups**
>
> Objective: Create two IAM users, a group with a policy, and add users to the group.
>
> Time: ~10 minutes

Step 1 — Set the account password policy

A strong password policy is the first thing you configure on any new AWS account.

```
# Navigate to IAM in the console, or use CLI:
aws iam update-account-password-policy \
  --minimum-password-length 12 \
  --require-symbols \
  --require-numbers \
  --require-uppercase-characters \
  --require-lowercase-characters \
  --allow-users-to-change-password \
  --max-password-age 90 \
  --password-reuse-prevention 5

# Verify the policy was applied
aws iam get-account-password-policy
```

Step 2 — Create an IAM group with a managed policy

```
# Create a group for data engineers
aws iam create-group --group-name sigma-data-engineers

# Attach AWS managed policies to the group
aws iam attach-group-policy \
  --group-name sigma-data-engineers \
  --policy-arn arn:aws:iam::aws:policy/ReadOnlyAccess

# Verify the attachment
aws iam list-attached-group-policies --group-name sigma-data-engineers
```

Step 3 — Create two IAM users

```
# Create user 1
aws iam create-user --user-name sigma-de-alice --tags
Key=Project,Value=sigma
```

```
# Create user 2
aws iam create-user --user-name sigma-de-bob --tags
Key=Project,Value=sigma

# Create console login profiles (password must meet policy requirements)
aws iam create-login-profile \
  --user-name sigma-de-alice \
  --password 'SigmaD@ta2024!' \
  --password-reset-required

aws iam create-login-profile \
  --user-name sigma-de-bob \
  --password 'SigmaD@ta2024!' \
  --password-reset-required
```

Step 4 — Add users to the group

```
aws iam add-user-to-group --user-name sigma-de-alice --group-name sigma-
data-engineers
aws iam add-user-to-group --user-name sigma-de-bob   --group-name sigma-
data-engineers

# Verify group membership
aws iam get-group --group-name sigma-data-engineers
```

What You Built

A production-pattern IAM setup: group → policy → users.

Users inherit permissions from the group, not from individual policy attachments.

This means when you need to change permissions for all data engineers, you change ONE group policy.

Password policy enforces MFA-ready security baseline across the account.

2.2 Lab — Custom IAM Policy for Data Engineering

LAB 2.2 — Write a Least-Privilege Policy for a DE Workload

Objective: Write a custom policy that grants exactly what a data pipeline needs — no more.

Time: ~12 minutes

Step 1 — Understand what the policy needs to allow

Pipeline Access Requirements

> The Sigma DataTech ingestion pipeline needs to:
>
> READ raw files from S3 bucket: sigma-raw-data-<account-id>
>
> WRITE processed files to S3: sigma-processed-<account-id>
>
> READ secrets from Secrets Manager (DB credentials)
>
> PUT CloudWatch log events (pipeline monitoring)
>
> Deny all S3 delete operations — pipeline should never delete source data

Step 2 — Create the policy document

```
# Create policy JSON file
cat > sigma-pipeline-policy.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadRawData",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::sigma-raw-data-*",
        "arn:aws:s3:::sigma-raw-data-*/*"
      ]
    },
    {
      "Sid": "WriteProcessedData",
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::sigma-processed-*",
        "arn:aws:s3:::sigma-processed-*/*"
      ]
    },
    {
      "Sid": "DenyAllDeletes",
      "Effect": "Deny",
      "Action": "s3:DeleteObject",
      "Resource": "*"
    },
    {
      "Sid": "ReadSecrets",
      "Effect": "Allow",
      "Action": ["secretsmanager:GetSecretValue"],
      "Resource": "arn:aws:secretsmanager:ap-south-1:*:secret:sigma/*"
    },
    {
      "Sid": "WriteLogs",
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
```

```
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:ap-south-1:*:log-group:/sigma/*"
    }
  ]
}
EOF
```

Step 3 — Create the policy in IAM

```
aws iam create-policy \
  --policy-name SigmaPipelinePolicy \
  --policy-document file://sigma-pipeline-policy.json \
  --description 'Least-privilege policy for Sigma DataTech ingestion
pipeline'

# Note the PolicyArn from the output — you will need it in Lab 2.3

# Validate the policy (check for syntax and logic errors)
aws iam get-policy --policy-arn
arn:aws:iam::<ACCOUNT_ID>:policy/SigmaPipelinePolicy
```

Key Policy Concepts

Effect: Deny always wins over Effect: Allow — even across different policies.

The DenyAllDeletes statement protects source data even if another policy allows deletes.

Resource ARNs use wildcards (*) to cover future bucket names matching the pattern.

The 'sigma/*' pattern on Secrets Manager scopes access to only Sigma project secrets.

2.3 Lab — IAM Roles and EC2 Instance Profiles

LAB 2.3 — Create a Role and Attach It to an EC2 Instance

Objective: Create an IAM role for EC2, attach the pipeline policy, verify access from the instance.

Time: ~12 minutes

Why: Services (EC2, Lambda, ECS) use roles — never hardcode credentials in application code.

Step 1 — Create a trust policy for EC2

A trust policy defines who can ASSUME this role. For EC2, the principal is the EC2 service.

```
cat > ec2-trust-policy.json << 'EOF'
{
```

```
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  }]
}
EOF
```

Step 2 — Create the role

```
aws iam create-role \
  --role-name SigmaPipelineEC2Role \
  --assume-role-policy-document file://ec2-trust-policy.json \
  --description 'Role for EC2 instances running Sigma DataTech pipelines'
\
  --tags Key=Project,Value=sigma

# Attach the custom pipeline policy
aws iam attach-role-policy \
  --role-name SigmaPipelineEC2Role \
  --policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/SigmaPipelinePolicy

# Attach CloudWatch agent policy (EC2 needs this to send metrics)
aws iam attach-role-policy \
  --role-name SigmaPipelineEC2Role \
  --policy-arn arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy
```

Step 3 — Create an instance profile and attach the role

```
# Instance profiles are the wrapper that lets EC2 use a role
aws iam create-instance-profile \
  --instance-profile-name SigmaPipelineEC2Profile

aws iam add-role-to-instance-profile \
  --instance-profile-name SigmaPipelineEC2Profile \
  --role-name SigmaPipelineEC2Role

# Verify the full role configuration
aws iam get-role --role-name SigmaPipelineEC2Role
aws iam list-attached-role-policies --role-name SigmaPipelineEC2Role
```

> **Role vs User — The Key Distinction**
>
> IAM User: A person. Has long-term access keys. Static credentials.
>
> IAM Role: An identity assumed temporarily. No permanent credentials.
>
> When EC2 assumes this role, AWS provides short-lived tokens (15 min – 12 hours).
>
> These rotate automatically — no credential rotation problem.
>
> Rule: Applications ALWAYS use roles. Humans use users (with MFA).

> Never put access keys in application code, config files, or environment variables on EC2.

2.4 Lab — IAM Policy Simulator and Access Testing

> **LAB 2.4 — Test Permissions Using the IAM Policy Simulator**
>
> Objective: Verify that the policy allows what it should and denies what it should.
>
> Time: ~8 minutes

Using the console Policy Simulator

1. Go to IAM → Policies → SigmaPipelinePolicy
2. Click Simulate policy in the top right
3. Under Actions, select S3 → GetObject — click Run simulation
4. Change the Resource to arn:aws:s3:::sigma-raw-data-test/file.csv
5. Expected result: Allowed
6. Now test S3 → DeleteObject on the same resource
7. Expected result: Denied (from the explicit Deny statement)
8. Test S3 → PutObject on arn:aws:s3:::sigma-processed-test/out.csv
9. Expected result: Allowed

Using the CLI to simulate

```
# Simulate specific actions using the CLI
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::<ACCOUNT_ID>:role/SigmaPipelineEC2Role
\
  --action-names s3:GetObject s3:DeleteObject s3:PutObject \
  --resource-arns \
    arn:aws:s3:::sigma-raw-data-123/data.csv \
    arn:aws:s3:::sigma-processed-123/out.csv

# Look for 'EvalDecision: allowed' vs 'EvalDecision: implicitDeny' or
'explicitDeny'
```

Command / Action	What it does
`aws iam create-user`	Create a new IAM user
`aws iam create-group`	Create a new IAM group
`aws iam attach-group-policy`	Attach a managed policy to a group
`aws iam create-policy`	Create a custom IAM policy from a JSON file
`aws iam create-role`	Create an IAM role with a trust policy
`aws iam attach-role-policy`	Attach a policy to a role

Command / Action	What it does
`aws iam create-instance-profile`	Create an EC2 instance profile
`aws iam simulate-principal-policy`	Simulate what actions a principal can perform
`aws sts get-caller-identity`	Verify which identity the CLI is using
`aws sts assume-role`	Temporarily assume a different IAM role

Module 3 — Networking for Data Platforms

You will build a complete, production-pattern VPC from scratch using the CLI. By the end you will have a three-tier network (public, private-app, private-data) that is the standard foundation for all data platform deployments.

Estimated lab time: 60 minutes across 5 labs.

Target Architecture

VPC: 10.0.0.0/16 (65,536 IPs — plenty of headroom)

Public Subnet A	10.0.1.0/24	ap-south-1a	NAT Gateway, Load Balancer
Public Subnet B	10.0.2.0/24	ap-south-1b	NAT Gateway (HA pair)
Private App A	10.0.10.0/24	ap-south-1a	EC2, ECS, Lambda
Private App B	10.0.11.0/24	ap-south-1b	EC2, ECS, Lambda
Private Data A	10.0.20.0/24	ap-south-1a	RDS, Redshift, ElastiCache
Private Data B	10.0.21.0/24	ap-south-1b	RDS, Redshift, ElastiCache

Internet Gateway: attached to VPC for public subnet internet access

NAT Gateway: in public subnet A — private subnets route outbound through it

S3 VPC Endpoint: private subnets access S3 without leaving AWS network

3.1 Lab — Create the VPC and Subnets

LAB 3.1 — Build the VPC Foundation

Objective: Create the VPC, all six subnets, and an Internet Gateway.

Time: ~15 minutes

Step 1 — Create the VPC

```
# Create the VPC
VPC_ID=$(aws ec2 create-vpc \
  --cidr-block 10.0.0.0/16 \
  --region ap-south-1 \
```

```
  --tag-specifications 'ResourceType=vpc,Tags=[
    {Key=Name,Value=sigma-vpc},
    {Key=Project,Value=sigma}]' \
  --query 'Vpc.VpcId' --output text)

echo "VPC created: $VPC_ID"

# Enable DNS resolution and DNS hostnames (required for RDS, EFS, etc.)
aws ec2 modify-vpc-attribute --vpc-id $VPC_ID --enable-dns-support
aws ec2 modify-vpc-attribute --vpc-id $VPC_ID --enable-dns-hostnames
```

Step 2 — Create all six subnets

```
# Public subnets
PUB_A=$(aws ec2 create-subnet --vpc-id $VPC_ID \
  --cidr-block 10.0.1.0/24 --availability-zone ap-south-1a \
  --tag-specifications 'ResourceType=subnet,Tags=[
    {Key=Name,Value=sigma-public-a},{Key=Tier,Value=public}]' \
  --query 'Subnet.SubnetId' --output text)

PUB_B=$(aws ec2 create-subnet --vpc-id $VPC_ID \
  --cidr-block 10.0.2.0/24 --availability-zone ap-south-1b \
  --tag-specifications 'ResourceType=subnet,Tags=[
    {Key=Name,Value=sigma-public-b},{Key=Tier,Value=public}]' \
  --query 'Subnet.SubnetId' --output text)

# Private app subnets
APP_A=$(aws ec2 create-subnet --vpc-id $VPC_ID \
  --cidr-block 10.0.10.0/24 --availability-zone ap-south-1a \
  --tag-specifications 'ResourceType=subnet,Tags=[
    {Key=Name,Value=sigma-private-app-a},{Key=Tier,Value=private-app}]' \
  --query 'Subnet.SubnetId' --output text)

APP_B=$(aws ec2 create-subnet --vpc-id $VPC_ID \
  --cidr-block 10.0.11.0/24 --availability-zone ap-south-1b \
  --tag-specifications 'ResourceType=subnet,Tags=[
    {Key=Name,Value=sigma-private-app-b},{Key=Tier,Value=private-app}]' \
  --query 'Subnet.SubnetId' --output text)

# Private data subnets
DATA_A=$(aws ec2 create-subnet --vpc-id $VPC_ID \
  --cidr-block 10.0.20.0/24 --availability-zone ap-south-1a \
  --tag-specifications 'ResourceType=subnet,Tags=[
    {Key=Name,Value=sigma-private-data-a},{Key=Tier,Value=private-data}]'
\
  --query 'Subnet.SubnetId' --output text)

DATA_B=$(aws ec2 create-subnet --vpc-id $VPC_ID \
  --cidr-block 10.0.21.0/24 --availability-zone ap-south-1b \
  --tag-specifications 'ResourceType=subnet,Tags=[
    {Key=Name,Value=sigma-private-data-b},{Key=Tier,Value=private-data}]'
\
  --query 'Subnet.SubnetId' --output text)

# Enable auto-assign public IP on public subnets only
aws ec2 modify-subnet-attribute --subnet-id $PUB_A --map-public-ip-on-
launch
```

```
aws ec2 modify-subnet-attribute --subnet-id $PUB_B --map-public-ip-on-
launch

# Save IDs for later labs
echo "PUB_A=$PUB_A  PUB_B=$PUB_B"
echo "APP_A=$APP_A  APP_B=$APP_B"
echo "DATA_A=$DATA_A  DATA_B=$DATA_B"
```

Step 3 — Create and attach an Internet Gateway

```
IGW_ID=$(aws ec2 create-internet-gateway \
  --tag-specifications 'ResourceType=internet-gateway,Tags=[
    {Key=Name,Value=sigma-igw},{Key=Project,Value=sigma}]' \
  --query 'InternetGateway.InternetGatewayId' --output text)

aws ec2 attach-internet-gateway --internet-gateway-id $IGW_ID --vpc-id
$VPC_ID
echo "IGW attached: $IGW_ID"
```

3.2 Lab — Route Tables

LAB 3.2 — Configure Routing for Public and Private Tiers

Objective: Create route tables, add routes, and associate subnets.

Time: ~12 minutes

Route Table Design

Public route table: 0.0.0.0/0 → Internet Gateway (traffic exits to internet)

Private route table: 0.0.0.0/0 → NAT Gateway (outbound only, no inbound)

Data route table: No internet route at all (completely isolated)

Every subnet that is NOT associated with a custom route table uses the VPC main route table.

Best practice: explicitly associate every subnet — never rely on the main route table.

Step 1 — Create a NAT Gateway for private subnet outbound access

```
# NAT Gateway needs an Elastic IP first
EIP_ID=$(aws ec2 allocate-address --domain vpc \
  --tag-specifications 'ResourceType=elastic-ip,Tags=[
    {Key=Name,Value=sigma-nat-eip},{Key=Project,Value=sigma}]' \
  --query 'AllocationId' --output text)
```

```
# Create NAT Gateway in the PUBLIC subnet (not private — this is a common
mistake)
NAT_ID=$(aws ec2 create-nat-gateway \
  --subnet-id $PUB_A \
  --allocation-id $EIP_ID \
  --tag-specifications 'ResourceType=natgateway,Tags=[
    {Key=Name,Value=sigma-nat},{Key=Project,Value=sigma}]' \
  --query 'NatGateway.NatGatewayId' --output text)

# Wait for NAT Gateway to become available (takes 1-2 minutes)
echo 'Waiting for NAT Gateway...'
aws ec2 wait nat-gateway-available --nat-gateway-ids $NAT_ID
echo "NAT Gateway ready: $NAT_ID"
```

> **NAT Gateway Placement**
>
> NAT Gateway MUST be in a PUBLIC subnet.
>
> A common mistake is creating it in a private subnet — it will have no internet access.
>
> The private subnets then route THROUGH the NAT Gateway via the public subnet.
>
> Cost note: NAT Gateway charges per hour + per GB processed. Delete it after the lab.

Step 2 — Create route tables

```
# Public route table
PUB_RT=$(aws ec2 create-route-table --vpc-id $VPC_ID \
  --tag-specifications 'ResourceType=route-table,Tags=[
    {Key=Name,Value=sigma-rt-public}]' \
  --query 'RouteTable.RouteTableId' --output text)

# Add route to internet via IGW
aws ec2 create-route --route-table-id $PUB_RT \
  --destination-cidr-block 0.0.0.0/0 --gateway-id $IGW_ID

# Private app route table
APP_RT=$(aws ec2 create-route-table --vpc-id $VPC_ID \
  --tag-specifications 'ResourceType=route-table,Tags=[
    {Key=Name,Value=sigma-rt-private-app}]' \
  --query 'RouteTable.RouteTableId' --output text)

# Add outbound route via NAT Gateway
aws ec2 create-route --route-table-id $APP_RT \
  --destination-cidr-block 0.0.0.0/0 --nat-gateway-id $NAT_ID

# Private data route table — NO internet route
DATA_RT=$(aws ec2 create-route-table --vpc-id $VPC_ID \
  --tag-specifications 'ResourceType=route-table,Tags=[
    {Key=Name,Value=sigma-rt-private-data}]' \
  --query 'RouteTable.RouteTableId' --output text)
```

Step 3 — Associate subnets with route tables

```
# Associate public subnets with public route table
aws ec2 associate-route-table --subnet-id $PUB_A --route-table-id $PUB_RT
aws ec2 associate-route-table --subnet-id $PUB_B --route-table-id $PUB_RT

# Associate private app subnets
aws ec2 associate-route-table --subnet-id $APP_A --route-table-id $APP_RT
aws ec2 associate-route-table --subnet-id $APP_B --route-table-id $APP_RT

# Associate private data subnets
aws ec2 associate-route-table --subnet-id $DATA_A --route-table-id
$DATA_RT
aws ec2 associate-route-table --subnet-id $DATA_B --route-table-id
$DATA_RT

# Verify routing — check each route table has the correct routes
aws ec2 describe-route-tables --route-table-ids $PUB_RT $APP_RT $DATA_RT
\
  --query
'RouteTables[*].[Tags[?Key==`Name`].Value|[0],Routes[*].DestinationCidrBl
ock]'
```

3.3 Lab — Security Groups

LAB 3.3 — Create Security Groups for Each Tier

Objective: Create four security groups following the principle of least privilege.

Time: ~10 minutes

Security Group Design

Bastion SG: Inbound SSH (22) from your IP only. Outbound: all.

App SG: Inbound SSH from Bastion SG only. Outbound: DB SG on 5432.

DB SG: Inbound PostgreSQL (5432) from App SG only. No outbound.

Pipeline SG: Inbound HTTPS (443) from App SG. Outbound: S3 (via VPC endpoint).

Key pattern: reference Security Group IDs in rules, NOT IP ranges.

This means permissions follow the security group, not a changing IP address.

Step 1 — Create the security groups

```
# Bastion host security group
SG_BASTION=$(aws ec2 create-security-group \
  --group-name sigma-sg-bastion \
  --description 'Bastion host — SSH access for admin entry' \
```

```
  --vpc-id $VPC_ID \
  --tag-specifications 'ResourceType=security-group,Tags=[
    {Key=Name,Value=sigma-sg-bastion},{Key=Project,Value=sigma}]' \
  --query 'GroupId' --output text)

# App tier security group
SG_APP=$(aws ec2 create-security-group \
  --group-name sigma-sg-app \
  --description 'App tier — receives traffic from bastion and load
balancer' \
  --vpc-id $VPC_ID \
  --tag-specifications 'ResourceType=security-group,Tags=[
    {Key=Name,Value=sigma-sg-app},{Key=Project,Value=sigma}]' \
  --query 'GroupId' --output text)

# Database security group
SG_DB=$(aws ec2 create-security-group \
  --group-name sigma-sg-database \
  --description 'Data tier — RDS and Redshift access from app tier only'
\
  --vpc-id $VPC_ID \
  --tag-specifications 'ResourceType=security-group,Tags=[
    {Key=Name,Value=sigma-sg-database},{Key=Project,Value=sigma}]' \
  --query 'GroupId' --output text)
```

Step 2 — Add ingress rules

```
# Get your current IP for bastion access
MY_IP=$(curl -s https://checkip.amazonaws.com)/32
echo "Your IP: $MY_IP"

# Bastion: allow SSH from your IP only
aws ec2 authorize-security-group-ingress \
  --group-id $SG_BASTION \
  --protocol tcp --port 22 --cidr $MY_IP

# App tier: allow SSH from bastion security group (not IP range)
aws ec2 authorize-security-group-ingress \
  --group-id $SG_APP \
  --protocol tcp --port 22 --source-group $SG_BASTION

# App tier: allow HTTPS from anywhere (web application)
aws ec2 authorize-security-group-ingress \
  --group-id $SG_APP \
  --protocol tcp --port 443 --cidr 0.0.0.0/0

# Database: allow PostgreSQL from app security group only
aws ec2 authorize-security-group-ingress \
  --group-id $SG_DB \
  --protocol tcp --port 5432 --source-group $SG_APP

# Database: allow Redshift port from app security group
aws ec2 authorize-security-group-ingress \
  --group-id $SG_DB \
  --protocol tcp --port 5439 --source-group $SG_APP

# Verify all security group rules
aws ec2 describe-security-groups \
```

```
  --group-ids $SG_BASTION $SG_APP $SG_DB \
  --query 'SecurityGroups[*].[GroupName,IpPermissions]'
```

3.4 Lab — VPC Endpoint for S3

LAB 3.4 — Create an S3 Gateway Endpoint

Objective: Allow private subnets to access S3 without internet traffic.

Time: ~8 minutes

Why VPC Endpoints Matter for Data Engineering

Without a VPC endpoint:

EC2/EMR in private subnet → NAT Gateway → Internet → S3

Cost: NAT Gateway processes every byte you read from S3.

Risk: Data leaves the AWS network — potential compliance issue.

With a Gateway endpoint (free for S3):

EC2/EMR in private subnet → VPC Endpoint → S3 (stays within AWS)

Cost: Zero. Gateway endpoints are free.

Risk: None. Traffic never leaves the AWS network.

For a data platform reading terabytes from S3, this is not optional.

Step 1 — Create the S3 Gateway endpoint

```
# Create S3 Gateway endpoint — attach to all three private route tables
aws ec2 create-vpc-endpoint \
  --vpc-id $VPC_ID \
  --vpc-endpoint-type Gateway \
  --service-name com.amazonaws.ap-south-1.s3 \
  --route-table-ids $APP_RT $DATA_RT \
  --policy-document '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Principal": "*",
      "Action": ["s3:GetObject","s3:PutObject","s3:ListBucket"],
      "Resource": ["arn:aws:s3:::sigma-*","arn:aws:s3:::sigma-*/*"]
    }]
```

```
  }' \
  --tag-specifications 'ResourceType=vpc-endpoint,Tags=[
    {Key=Name,Value=sigma-vpce-s3},{Key=Project,Value=sigma}]'
```

Step 2 — Verify the endpoint is in your route tables

```
# The endpoint adds a prefix list route automatically
aws ec2 describe-route-tables --route-table-ids $APP_RT \
  --query 'RouteTables[*].Routes[?GatewayId!=`local`]'
# You should see a route with DestinationPrefixListId starting with 'pl-'
```

Step 3 (Optional) — Create an Interface endpoint for Secrets Manager

```
# Interface endpoints cost money (~$7/month per AZ) — create and delete
promptly
aws ec2 create-vpc-endpoint \
  --vpc-id $VPC_ID \
  --vpc-endpoint-type Interface \
  --service-name com.amazonaws.ap-south-1.secretsmanager \
  --subnet-ids $APP_A $APP_B \
  --security-group-ids $SG_APP \
  --private-dns-enabled \
  --tag-specifications 'ResourceType=vpc-endpoint,Tags=[
    {Key=Name,Value=sigma-vpce-
secretsmanager},{Key=Project,Value=sigma}]'

# DELETE after verifying to avoid unnecessary charges
# aws ec2 delete-vpc-endpoints --vpc-endpoint-ids <endpoint-id>
```

3.5 Lab — Network ACLs and Security Group vs NACL Comparison

LAB 3.5 — Create a NACL for the Data Tier and Understand the Difference

Objective: Create a restrictive NACL for the data subnet, understand stateful vs stateless.

Time: ~10 minutes

Property	Security Group	Network ACL
Level	Instance / ENI	Subnet
State	Stateful (return traffic automatic)	Stateless (both directions explicit)
Rules	Allow only	Allow and Deny
Rule evaluation	All rules evaluated together	Rules evaluated in order (lowest number first)
Default	Deny all inbound, allow all outbound	Allow all inbound and outbound
Best for	Fine-grained instance access control	Subnet-level blast radius control

Step 1 — Create a NACL for the data subnet

```
NACL_ID=$(aws ec2 create-network-acl \
  --vpc-id $VPC_ID \
  --tag-specifications 'ResourceType=network-acl,Tags=[
    {Key=Name,Value=sigma-nacl-data}]' \
  --query 'NetworkAcl.NetworkAclId' --output text)
```

Step 2 — Add inbound rules

```
# Rule 100: Allow PostgreSQL from private app CIDR
aws ec2 create-network-acl-entry \
  --network-acl-id $NACL_ID --ingress --rule-number 100 \
  --protocol tcp --port-range From=5432,To=5432 \
  --cidr-block 10.0.10.0/23 --rule-action allow

# Rule 110: Allow ephemeral return traffic (1024-65535)
aws ec2 create-network-acl-entry \
  --network-acl-id $NACL_ID --ingress --rule-number 110 \
  --protocol tcp --port-range From=1024,To=65535 \
  --cidr-block 10.0.10.0/23 --rule-action allow

# Rule 32766: Deny everything else (explicit)
aws ec2 create-network-acl-entry \
  --network-acl-id $NACL_ID --ingress --rule-number 32766 \
  --protocol -1 --cidr-block 0.0.0.0/0 --rule-action deny
```

Step 3 — Add outbound rules and associate with data subnets

```
# Outbound: allow ephemeral ports back to app tier
aws ec2 create-network-acl-entry \
  --network-acl-id $NACL_ID --egress --rule-number 100 \
  --protocol tcp --port-range From=1024,To=65535 \
  --cidr-block 10.0.10.0/23 --rule-action allow

# Associate NACL with data subnets (replaces the default NACL)
# First find the current associations
aws ec2 describe-network-acls --filters Name=vpc-id,Values=$VPC_ID \
  --query
'NetworkAcls[*].Associations[*].[SubnetId,NetworkAclAssociationId]'

# Replace the association for each data subnet
# (Use the AssociationId from the output above)
aws ec2 replace-network-acl-association \
  --association-id <DATA_A_ASSOC_ID> --network-acl-id $NACL_ID
aws ec2 replace-network-acl-association \
  --association-id <DATA_B_ASSOC_ID> --network-acl-id $NACL_ID
```

NACLs Are Stateless — Remember Ephemeral Ports

Security Groups are STATEFUL: allow inbound on 5432, return traffic is automatic.

NACLs are STATELESS: you must explicitly allow the return traffic.

> TCP return traffic uses ephemeral ports (1024-65535) — always include this in NACL outbound rules.
>
> Forgetting ephemeral ports is the #1 NACL mistake and causes mysterious connectivity failures.

Command / Action	What it does
`aws ec2 create-vpc`	Create a new VPC with a CIDR block
`aws ec2 create-subnet`	Create a subnet in a specific AZ
`aws ec2 create-internet-gateway`	Create an Internet Gateway
`aws ec2 attach-internet-gateway`	Attach IGW to VPC
`aws ec2 allocate-address`	Allocate an Elastic IP for NAT Gateway
`aws ec2 create-nat-gateway`	Create a NAT Gateway in a public subnet
`aws ec2 create-route-table`	Create a custom route table
`aws ec2 create-route`	Add a route to a route table
`aws ec2 associate-route-table`	Associate a subnet with a route table
`aws ec2 create-security-group`	Create a security group
`aws ec2 authorize-security-group-ingress`	Add an inbound rule to a security group
`aws ec2 create-vpc-endpoint`	Create a VPC endpoint (Gateway or Interface)
`aws ec2 create-network-acl`	Create a Network ACL for a VPC
`aws ec2 create-network-acl-entry`	Add a rule to a Network ACL

Module 4 — Infrastructure as Code & CI/CD

CloudFormation turns everything you built manually in Module 3 into repeatable, version-controlled infrastructure. You will template the VPC, experience a live rollback, and set up a CodePipeline that deploys on every Git push.

Estimated lab time: 60 minutes across 4 labs.

4.1 Lab — Your First CloudFormation Stack

LAB 4.1 — Deploy a VPC with CloudFormation

Objective: Write a CloudFormation template that creates a VPC with two subnets.

Time: ~15 minutes

Why this approach: Everything built manually in Lab 3 gets destroyed and rebuilt in under 2 minutes.

Step 1 — Write the CloudFormation template

```
# Create the template file
cat > sigma-network.yaml << 'EOF'
AWSTemplateFormatVersion: '2010-09-09'
Description: 'Sigma DataTech — Network Foundation Stack'

Parameters:
  Environment:
    Type: String
    Default: dev
    AllowedValues: [dev, staging, prod]
    Description: Deployment environment
  VpcCidr:
    Type: String
    Default: '10.0.0.0/16'
    Description: CIDR block for the VPC

Resources:
  SigmaVPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: !Ref VpcCidr
      EnableDnsSupport: true
      EnableDnsHostnames: true
      Tags:
        - Key: Name
          Value: !Sub 'sigma-vpc-${Environment}'
        - Key: Project
          Value: sigma
        - Key: Environment
          Value: !Ref Environment
```

```yaml
  InternetGateway:
    Type: AWS::EC2::InternetGateway
    Properties:
      Tags:
        - Key: Name
          Value: !Sub 'sigma-igw-${Environment}'

  IGWAttachment:
    Type: AWS::EC2::VPCGatewayAttachment
    Properties:
      VpcId: !Ref SigmaVPC
      InternetGatewayId: !Ref InternetGateway

  PublicSubnetA:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref SigmaVPC
      CidrBlock: '10.0.1.0/24'
      AvailabilityZone: !Select [0, !GetAZs '']
      MapPublicIpOnLaunch: true
      Tags:
        - Key: Name
          Value: !Sub 'sigma-public-a-${Environment}'
        - Key: Tier
          Value: public

  PrivateSubnetA:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref SigmaVPC
      CidrBlock: '10.0.10.0/24'
      AvailabilityZone: !Select [0, !GetAZs '']
      MapPublicIpOnLaunch: false
      Tags:
        - Key: Name
          Value: !Sub 'sigma-private-app-a-${Environment}'
        - Key: Tier
          Value: private-app

  PublicRouteTable:
    Type: AWS::EC2::RouteTable
    Properties:
      VpcId: !Ref SigmaVPC
      Tags:
        - Key: Name
          Value: !Sub 'sigma-rt-public-${Environment}'

  PublicRoute:
    Type: AWS::EC2::Route
    DependsOn: IGWAttachment
    Properties:
      RouteTableId: !Ref PublicRouteTable
      DestinationCidrBlock: '0.0.0.0/0'
      GatewayId: !Ref InternetGateway

  PublicSubnetAAssoc:
    Type: AWS::EC2::SubnetRouteTableAssociation
    Properties:
```

```
      SubnetId: !Ref PublicSubnetA
      RouteTableId: !Ref PublicRouteTable

Outputs:
  VpcId:
    Description: VPC ID
    Value: !Ref SigmaVPC
    Export:
      Name: !Sub '${AWS::StackName}-VpcId'
  PublicSubnetAId:
    Description: Public Subnet A ID
    Value: !Ref PublicSubnetA
    Export:
      Name: !Sub '${AWS::StackName}-PublicSubnetA'
EOF
```

Step 2 — Validate the template before deploying

```
aws cloudformation validate-template --template-body file://sigma-
network.yaml
# If the template has errors, this will report them before any resources
are created
```

Step 3 — Deploy the stack

```
aws cloudformation create-stack \
  --stack-name sigma-network-dev \
  --template-body file://sigma-network.yaml \
  --parameters ParameterKey=Environment,ParameterValue=dev \
  --tags Key=Project,Value=sigma Key=ManagedBy,Value=cloudformation

# Watch the stack build in real time
aws cloudformation wait stack-create-complete --stack-name sigma-network-
dev

# Alternatively, watch events as they happen:
aws cloudformation describe-stack-events --stack-name sigma-network-dev \
  --query 'StackEvents[*].[Timestamp,ResourceStatus,LogicalResourceId]' \
  --output table
```

Step 4 — Verify the stack and read outputs

```
# Check stack status
aws cloudformation describe-stacks --stack-name sigma-network-dev \
  --query 'Stacks[0].[StackStatus,Outputs]'

# Get the exported VPC ID
aws cloudformation list-exports \
  --query 'Exports[?Name==`sigma-network-dev-VpcId`].Value'
```

What CloudFormation Gave You

Repeatability: run the same template in dev, staging, and prod — identical infrastructure.

Rollback: if any resource fails to create, CloudFormation rolls back the entire stack automatically.

Audit trail: every change to the stack is logged with who made it and when.

Drift detection: CloudFormation can tell you if someone manually changed a resource.

Outputs: the VpcId export lets other stacks reference this VPC without hardcoding IDs.

4.2 Lab — Stack Update, Change Sets, and Rollback

LAB 4.2 — Update a Stack Using a Change Set

Objective: Update the stack safely using a change set, then deliberately cause a rollback.

Time: ~12 minutes

Step 1 — Add a second subnet to the template

Add the following resource to your sigma-network.yaml under Resources (after PrivateSubnetA):

```
# Add this to the Resources section of sigma-network.yaml
  PrivateSubnetB:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref SigmaVPC
      CidrBlock: '10.0.11.0/24'
      AvailabilityZone: !Select [1, !GetAZs '']
      MapPublicIpOnLaunch: false
      Tags:
        - Key: Name
          Value: !Sub 'sigma-private-app-b-${Environment}'
        - Key: Tier
          Value: private-app
```

Step 2 — Preview the change with a Change Set

```
# Create a change set — does NOT deploy yet
aws cloudformation create-change-set \
  --stack-name sigma-network-dev \
  --template-body file://sigma-network.yaml \
  --change-set-name add-private-subnet-b \
  --parameters ParameterKey=Environment,ParameterValue=dev

# Review what will change BEFORE applying
aws cloudformation describe-change-set \
  --stack-name sigma-network-dev \
  --change-set-name add-private-subnet-b \
  --query
'Changes[*].[Type,ResourceChange.Action,ResourceChange.LogicalResourceId]
'
# Expected: Add | PrivateSubnetB
```

Step 3 — Execute the change set

```
aws cloudformation execute-change-set \
  --stack-name sigma-network-dev \
  --change-set-name add-private-subnet-b

aws cloudformation wait stack-update-complete --stack-name sigma-network-
dev
echo "Stack updated successfully"
```

Step 4 — Trigger a deliberate rollback

Now we will intentionally cause a failure to see rollback in action. Add a subnet with a CIDR that overlaps an existing one:

> **Rollback in Action**
>
> CloudFormation will attempt to create the subnet with the overlapping CIDR.
>
> EC2 will reject it — CIDR conflict.
>
> CloudFormation will automatically roll back ALL changes made during this update.
>
> The stack returns to its last known good state.
>
> This is the value of infrastructure as code — failed changes cannot leave you in a broken state.

```
# Add this broken resource (duplicate CIDR 10.0.11.0/24 — will fail)
# Add to sigma-network.yaml under Resources:
  BrokenSubnet:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref SigmaVPC
      CidrBlock: '10.0.11.0/24'   # This CIDR already exists — will cause
failure
      AvailabilityZone: !Select [1, !GetAZs '']

# Deploy the broken update
aws cloudformation update-stack \
  --stack-name sigma-network-dev \
  --template-body file://sigma-network.yaml \
  --parameters ParameterKey=Environment,ParameterValue=dev

# Watch the rollback happen — check events
aws cloudformation describe-stack-events --stack-name sigma-network-dev \
  --query
'StackEvents[*].[ResourceStatus,LogicalResourceId,ResourceStatusReason]'
\
  --output table
# You will see:  ROLLBACK_IN_PROGRESS → UPDATE_ROLLBACK_COMPLETE
```

4.3 Lab — Cross-Stack References and Nested Stacks

LAB 4.3 — Export Values and Use Them in a Second Stack

Objective: Create a security group stack that imports the VPC ID from the network stack.

Time: ~10 minutes

Step 1 — Create the security group template

```
cat > sigma-security.yaml << 'EOF'
AWSTemplateFormatVersion: '2010-09-09'
Description: 'Sigma DataTech — Security Groups Stack'

Parameters:
  NetworkStackName:
    Type: String
    Default: sigma-network-dev
    Description: Name of the network stack to import VPC ID from

Resources:
  AppSecurityGroup:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: 'App tier security group'
      VpcId:
        Fn::ImportValue: !Sub '${NetworkStackName}-VpcId'
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 443
          ToPort: 443
          CidrIp: '0.0.0.0/0'
          Description: 'HTTPS from anywhere'
      Tags:
        - Key: Name
          Value: sigma-sg-app
        - Key: Project
          Value: sigma

  DatabaseSecurityGroup:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: 'Database tier security group'
      VpcId:
        Fn::ImportValue: !Sub '${NetworkStackName}-VpcId'
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 5432
          ToPort: 5432
          SourceSecurityGroupId: !Ref AppSecurityGroup
          Description: 'PostgreSQL from app tier only'
      Tags:
        - Key: Name
          Value: sigma-sg-database
        - Key: Project
          Value: sigma
```

```
Outputs:
  AppSgId:
    Value: !Ref AppSecurityGroup
    Export:
      Name: !Sub '${AWS::StackName}-AppSgId'
  DbSgId:
    Value: !Ref DatabaseSecurityGroup
    Export:
      Name: !Sub '${AWS::StackName}-DbSgId'
EOF
```

Step 2 — Deploy the security group stack

```
aws cloudformation create-stack \
  --stack-name sigma-security-dev \
  --template-body file://sigma-security.yaml \
  --parameters ParameterKey=NetworkStackName,ParameterValue=sigma-
network-dev \
  --tags Key=Project,Value=sigma

aws cloudformation wait stack-create-complete --stack-name sigma-
security-dev
aws cloudformation describe-stacks --stack-name sigma-security-dev \
  --query 'Stacks[0].Outputs'
```

Why Cross-Stack References?

Splitting infrastructure into multiple stacks prevents a single monolithic template.

Network stack: VPC, subnets, routing — rarely changes

Security stack: Security groups — changes with new services

Database stack: RDS, Redshift — changes with schema updates

App stack: EC2, ECS, Lambda — changes with every deployment

When the app stack deploys 20 times per day, it does not touch the network stack.

Import/Export creates a hard dependency — AWS prevents deleting an exported value

if another stack is importing it.

4.4 Lab — CI/CD Pipeline with CodePipeline

LAB 4.4 — Create a Pipeline That Deploys CloudFormation on Every Push

Objective: Set up CodeCommit → CodeBuild → CodePipeline to auto-deploy infrastructure.

Time: ~20 minutes

Step 1 — Create a CodeCommit repository

```
aws codecommit create-repository \
  --repository-name sigma-infra \
  --repository-description 'Sigma DataTech infrastructure as code' \
  --tags Project=sigma

# Get the clone URL
aws codecommit get-repository --repository-name sigma-infra \
  --query 'repositoryMetadata.cloneUrlHttp'

# Clone the repository
git clone https://git-codecommit.ap-south-1.amazonaws.com/v1/repos/sigma-
infra
cd sigma-infra

# Configure Git credentials for CodeCommit (if not already done)
# In the console: IAM → Your user → Security credentials → HTTPS Git
credentials
# Download the credentials and use them when prompted
```

Step 2 — Commit the CloudFormation templates

```
# Copy templates into the repo
cp ../sigma-network.yaml .
cp ../sigma-security.yaml .

# Create a buildspec for CodeBuild
cat > buildspec.yml << 'EOF'
version: 0.2
phases:
  install:
    commands:
      - pip install cfn-lint
  pre_build:
    commands:
      - echo 'Validating CloudFormation templates...'
      - cfn-lint sigma-network.yaml
      - cfn-lint sigma-security.yaml
      - aws cloudformation validate-template --template-body
file://sigma-network.yaml
      - aws cloudformation validate-template --template-body
file://sigma-security.yaml
  build:
    commands:
      - echo 'Templates validated successfully'
      - aws s3 cp sigma-network.yaml s3://sigma-cfn-templates-
${AWS_ACCOUNT_ID}/
      - aws s3 cp sigma-security.yaml s3://sigma-cfn-templates-
${AWS_ACCOUNT_ID}/
artifacts:
  files:
    - sigma-network.yaml
    - sigma-security.yaml
EOF
```

```
# Initial commit and push
git add .
git commit -m "feat: Add network and security CloudFormation templates"
git push origin main
```

Step 3 — Create the IAM role for CodePipeline

```
# Trust policy for CodePipeline service
cat > codepipeline-trust.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Service": "codepipeline.amazonaws.com"},
    "Action": "sts:AssumeRole"
  }]
}
EOF

aws iam create-role \
  --role-name SigmaCodePipelineRole \
  --assume-role-policy-document file://codepipeline-trust.json

aws iam attach-role-policy \
  --role-name SigmaCodePipelineRole \
  --policy-arn arn:aws:iam::aws:policy/AWSCodePipelineFullAccess
aws iam attach-role-policy \
  --role-name SigmaCodePipelineRole \
  --policy-arn arn:aws:iam::aws:policy/AWSCloudFormationFullAccess
aws iam attach-role-policy \
  --role-name SigmaCodePipelineRole \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3FullAccess
```

Step 4 — Create an S3 bucket for pipeline artifacts

```
# Replace ACCOUNT_ID with your actual 12-digit account ID
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

aws s3 mb s3://sigma-pipeline-artifacts-${ACCOUNT_ID} --region ap-south-1
aws s3 mb s3://sigma-cfn-templates-${ACCOUNT_ID}     --region ap-south-1

# Enable versioning on artifact bucket
aws s3api put-bucket-versioning \
  --bucket sigma-pipeline-artifacts-${ACCOUNT_ID} \
  --versioning-configuration Status=Enabled
```

Step 5 — Create the CodePipeline

```
cat > pipeline-definition.json << EOF
{
  "pipeline": {
    "name": "sigma-infra-pipeline",
    "roleArn": "arn:aws:iam::${ACCOUNT_ID}:role/SigmaCodePipelineRole",
    "artifactStore": {
      "type": "S3",
      "location": "sigma-pipeline-artifacts-${ACCOUNT_ID}"
```

```
    },
    "stages": [
      {
        "name": "Source",
        "actions": [{
          "name": "CodeCommitSource",
          "actionTypeId": {
            "category": "Source", "owner": "AWS",
            "provider": "CodeCommit", "version": "1"
          },
          "configuration": {
            "RepositoryName": "sigma-infra",
            "BranchName": "main",
            "PollForSourceChanges": "true"
          },
          "outputArtifacts": [{"name": "SourceOutput"}]
        }]
      },
      {
        "name": "Validate",
        "actions": [{
          "name": "ValidateTemplates",
          "actionTypeId": {
            "category": "Build", "owner": "AWS",
            "provider": "CodeBuild", "version": "1"
          },
          "configuration": {"ProjectName": "sigma-cfn-validate"},
          "inputArtifacts": [{"name": "SourceOutput"}],
          "outputArtifacts": [{"name": "BuildOutput"}]
        }]
      },
      {
        "name": "Deploy",
        "actions": [{
          "name": "DeployNetworkStack",
          "actionTypeId": {
            "category": "Deploy", "owner": "AWS",
            "provider": "CloudFormation", "version": "1"
          },
          "configuration": {
            "ActionMode": "CREATE_UPDATE",
            "StackName": "sigma-network-dev",
            "TemplatePath": "BuildOutput::sigma-network.yaml",
            "Capabilities": "CAPABILITY_IAM",
            "RoleArn":
"arn:aws:iam::${ACCOUNT_ID}:role/SigmaCodePipelineRole"
          },
          "inputArtifacts": [{"name": "BuildOutput"}]
        }]
      }
    ]
  }
}
EOF

# Substitute actual account ID
sed -i "s/\${ACCOUNT_ID}/${ACCOUNT_ID}/g" pipeline-definition.json
```

```
# Create the pipeline
aws codepipeline create-pipeline --cli-input-json file://pipeline-
definition.json
```

Step 6 — Trigger and observe the pipeline

```
# Make a small change to trigger the pipeline
echo '# Updated by pipeline test' >> sigma-network.yaml
git add . && git commit -m "chore: Trigger pipeline test"
git push

# Watch the pipeline in the console: CodePipeline → sigma-infra-pipeline
# Or use CLI to check pipeline status:
aws codepipeline get-pipeline-state --name sigma-infra-pipeline \
  --query 'stageStates[*].[stageName,latestExecution.status]'
```

What You Built

A complete GitOps workflow: push to Git → validate → deploy infrastructure.

Any change to the CloudFormation templates is automatically validated and deployed.

Failed deployments roll back automatically — no manual intervention required.

Every infrastructure change has a full audit trail in both CodeCommit and CloudFormation.

Command / Action	What it does
`aws cloudformation validate-template`	Validate template syntax before deploying
`aws cloudformation create-stack`	Deploy a new CloudFormation stack
`aws cloudformation update-stack`	Update an existing stack
`aws cloudformation create-change-set`	Preview changes before applying them
`aws cloudformation execute-change-set`	Apply a previously created change set
`aws cloudformation wait stack-create-complete`	Block until stack creation is done
`aws cloudformation describe-stack-events`	View all events including failures
`aws cloudformation describe-stacks`	Get stack status and outputs
`aws cloudformation list-exports`	List all cross-stack exported values
`aws codecommit create-repository`	Create a new CodeCommit Git repository
`aws codepipeline create-pipeline`	Create a CI/CD pipeline

Command / Action	What it does
`aws codepipeline get-pipeline-state`	Check current pipeline execution status

Lab Cleanup — Delete All Resources

Always clean up after lab sessions. AWS charges for idle resources. Run these commands at the end of Day 2.

> **Clean Up in This Order**
>
> Delete stacks and pipelines first — they may depend on other resources.
>
> Delete VPC resources in reverse creation order (endpoints, subnets, IGW, then VPC).
>
> NAT Gateways accrue hourly charges — delete these first if you are pausing.

CloudFormation Stacks

```
# Delete in reverse dependency order — security stack first (imports from network)
aws cloudformation delete-stack --stack-name sigma-security-dev
aws cloudformation wait stack-delete-complete --stack-name sigma-security-dev

aws cloudformation delete-stack --stack-name sigma-network-dev
aws cloudformation wait stack-delete-complete --stack-name sigma-network-dev
```

CI/CD Resources

```
aws codepipeline delete-pipeline --name sigma-infra-pipeline
aws codecommit delete-repository --repository-name sigma-infra
aws s3 rb s3://sigma-pipeline-artifacts-${ACCOUNT_ID} --force
aws s3 rb s3://sigma-cfn-templates-${ACCOUNT_ID} --force
```

VPC Resources (if built manually in Module 3)

```
# Delete NAT Gateway first (accrues charges per hour)
aws ec2 delete-nat-gateway --nat-gateway-id $NAT_ID
aws ec2 wait nat-gateway-deleted --nat-gateway-ids $NAT_ID

# Release Elastic IP
aws ec2 release-address --allocation-id $EIP_ID

# Delete VPC endpoints
aws ec2 describe-vpc-endpoints --filters Name=vpc-id,Values=$VPC_ID \
  --query 'VpcEndpoints[*].VpcEndpointId' --output text | \
  xargs aws ec2 delete-vpc-endpoints --vpc-endpoint-ids

# Delete security groups (except default)
aws ec2 delete-security-group --group-id $SG_DB
aws ec2 delete-security-group --group-id $SG_APP
aws ec2 delete-security-group --group-id $SG_BASTION
```

```
# Delete subnets
for SN in $PUB_A $PUB_B $APP_A $APP_B $DATA_A $DATA_B; do
  aws ec2 delete-subnet --subnet-id $SN
done

# Detach and delete Internet Gateway
aws ec2 detach-internet-gateway --internet-gateway-id $IGW_ID --vpc-id
$VPC_ID
aws ec2 delete-internet-gateway --internet-gateway-id $IGW_ID

# Delete route tables (cannot delete main route table)
for RT in $PUB_RT $APP_RT $DATA_RT; do
  aws ec2 delete-route-table --route-table-id $RT
done

# Delete VPC
aws ec2 delete-vpc --vpc-id $VPC_ID
```

IAM Resources

```
# Detach policies before deleting role
aws iam detach-role-policy --role-name SigmaPipelineEC2Role \
  --policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/SigmaPipelinePolicy
aws iam detach-role-policy --role-name SigmaPipelineEC2Role \
  --policy-arn arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy
aws iam remove-role-from-instance-profile \
  --instance-profile-name SigmaPipelineEC2Profile \
  --role-name SigmaPipelineEC2Role
aws iam delete-instance-profile --instance-profile-name
SigmaPipelineEC2Profile
aws iam delete-role --role-name SigmaPipelineEC2Role
aws iam delete-policy --policy-arn
arn:aws:iam::<ACCOUNT_ID>:policy/SigmaPipelinePolicy

# Remove users from group and delete
aws iam remove-user-from-group --user-name sigma-de-alice --group-name
sigma-data-engineers
aws iam remove-user-from-group --user-name sigma-de-bob   --group-name
sigma-data-engineers
aws iam delete-login-profile --user-name sigma-de-alice
aws iam delete-login-profile --user-name sigma-de-bob
aws iam delete-user --user-name sigma-de-alice
aws iam delete-user --user-name sigma-de-bob
aws iam detach-group-policy --group-name sigma-data-engineers \
  --policy-arn arn:aws:iam::aws:policy/ReadOnlyAccess
aws iam delete-group --group-name sigma-data-engineers
```

Day 2 Complete — Mission 02: GO CLOUD

You built a production-grade AWS environment from scratch:

Module 2: IAM users, groups, roles, least-privilege policies, instance profiles

Module 3: 3-tier VPC, routing, security groups, NACLs, VPC endpoints

Module 4: CloudFormation templates, change sets, rollback, cross-stack references, CI/CD

Tomorrow: Mission 03 — PIPE UP. AWS Data Engineering — Glue, Kinesis, Redshift, Athena.

Everything you built today is the foundation every data platform runs on.

You will use VPCs, IAM roles, and CloudFormation stacks every single day from here.